MAGENTO 2 · PREFERENCE DEMO

Codilar_PreferenceDemo

How the “Members” header logo works, a file-by-file code explanation, and the questions to expect when presenting this demo.

Module	`app/code/Codilar/PreferenceDemo`
Platform	Magento Open Source 2.4.9 · PHP 8.5 · Luma theme
Replaced class	`Magento\Theme\Block\Html\Header\Logo`
Replacement	`Codilar\PreferenceDemo\Block\Html\Header\Logo`
Scope	Storefront only (`etc/frontend/di.xml`)
Date	October 2026

1. What the demo does
2. Preference fundamentals
3. Request flow
4. Module structure
5. File-by-file code explanation
6. Tested scenarios
7. Install, configure and test
8. Design decisions and pitfalls
9. Known limitations
10. Expected questions and answers

1. What the demo does

The storefront header shows a different logo depending on who is looking:

- **Guests** see the normal Luma logo.
- **Logged-in customers** see a “Members” version: the same logo with an orange **MEMBERS** badge under the word LUMA, and “Members” as the image's alt text.

Guest (core logo.svg, 170px)



Logged in (logo-members.svg, 170px)



Logged in, enlarged



Left: what guests see. Middle: what logged-in customers see, at the real header size (170px wide). Right: the Members logo enlarged. Both logos have exactly the same dimensions, so the header does not shift on login.

This is done with a **preference**: Magento is told that whenever it needs the core header logo block, it should create our class instead. Our class extends the core one and changes only two methods.

#	Step	What happens in code
1	A storefront page is built.	The layout file `Magento_Theme::default.xml` declares a block named `logo` of class `Magento\Theme\Block\Html\Header\Logo`.
2	Magento creates that block.	The object manager looks up the class, finds our preference in `etc/frontend/di.xml`, and creates `Codilar\PreferenceDemo\Block\Html\Header\Logo` instead.
3	The logo template asks for the image.	`logo.phtml` calls `$block->getLogoSrc()` and `$block->getLogoAlt()`, which now run our versions.
4	Our block checks whether the visitor is logged in.	It reads `customer_logged_in` from Magento's HTTP context.
5	It returns the right logo.	Guest → `parent::getLogoSrc()` (normal logo). Logged in → URL of `logo-members.svg` from the module.

> **Nothing on the admin side was changed.** There were no admin code, configuration or template changes, and no core or theme files were edited. The logo set under Content → Design → Configuration still controls what guests see.

2. Preference fundamentals

A **preference** is a rule in `di.xml` that tells Magento's object manager which concrete class to create when a given class or interface is requested:

```
syntax
<preference for="Requested\Class\Or\Interface" type="Class\To\Create\Instead"/>
```

Magento uses preferences everywhere to connect interfaces to implementations. For example, core's own `di.xml` maps `Magento\Catalog\Api\ProductRepositoryInterface` to `Magento\Catalog\Model\ProductRepository`. A module can also use a preference on a **concrete class** to substitute its own subclass, as this demo does.

How the object manager uses it

1. Some code (here, the layout) asks the object manager for `Magento\Theme\Block\Html\Header\Logo`.
2. The object manager checks the merged DI configuration for the current area and finds the preference.
3. It builds `Codilar\PreferenceDemo\Block\Html\Header\Logo` instead, resolving **that class's** constructor, including our extra `HttpContext` dependency.
4. The caller receives our object. Because it extends the core class, it is still a valid `Logo` for every type check and template call.

Magento reports exactly this for the storefront area:

```
php bin/magento dev:di:info "Magento\Theme\Block\Html\Header\Logo" frontend
DI configuration for the class Magento\Theme\Block\Html\Header\Logo in the FRONTEND area

Preference: Codilar\PreferenceDemo\Block\Html\Header\Logo

Constructor Parameters:
| context           | Magento\Framework\View\Element\Template\Context   |               |
| fileStorageHelper | Magento\MediaStorage\Helper\File\Storage\Database |               |
| httpContext       | Magento\Framework\App\Http\Context                |               |
| data              |                                                   | <empty array> |
```

Run without `frontend`, the same command reports the **global** area, where the preference is still the core class. This confirms that the admin and other areas are unaffected.

Preference vs plugin vs observer

Technique	What it does	Conflicts
Preference	Replaces the whole class. Can override any public or protected method and add new dependencies.	Only **one** preference per class can win. Another module's preference for the same class silently replaces yours.
Plugin	Wraps one public method (before / around / after) of the existing class (see Codilar_CartPlugins).	Many plugins can coexist on the same method, ordered by `sortOrder`.
Observer	Reacts to an event the code already dispatches (see Codilar_EventDemo).	Many observers can coexist.

Rule of thumb: prefer an observer or plugin. Use a preference for a concrete class only when you need what they can't do, e.g. change protected methods or several closely related methods at once. For interfaces, preferences are the normal and correct tool.

Why a preference fits this demo

- The two methods (`getLogoSrc()`, `getLogoAlt()`) are public, so after plugins could also do this job. The preference is used here to demonstrate the technique: substituting a subclass, overriding methods and calling `parent::`, and adding a constructor dependency.
- No event is dispatched when the logo is rendered, so an observer is not an option.
- Editing the core class or the template directly would be lost on upgrade. A preference keeps the change in our module.

3. Request flow

What happens when any storefront page is requested (Magento 2.4.9, full-page cache enabled).

Request arrives (any storefront page)
Magento\Customer\Model\App\Action\ContextPlugin::beforeExecute() copies the login state into the HTTP context: `httpContext->setValue('customer_logged_in', $customerSession->isLoggedIn(), false)`

▼

Full-page cache lookup
The cache key includes the HTTP context's vary string (`X-Magento-Vary` cookie / `Http\Context::getVaryString()`). Guests and logged-in customers therefore have **different** cache keys.

▼ cache HIT → stored page returned as-is (it already contains the right logo)
▼ cache MISS → page is built

Layout: <block class="Magento\Theme\Block\Html\Header\Logo" name="logo">
from Magento_Theme/view/frontend/layout/default.xml

▼ object manager creates the block

Preference (etc/frontend/di.xml)
Magento\Theme\Block\Html\Header\Logo → **Codilar\PreferenceDemo\Block\Html\Header\Logo**

▼

Layout generated → customer session depersonalised
For cacheable pages, `Customer\Model\Layout\DepersonalizePlugin::afterGenerateElements()` clears the customer session **before** blocks render. The HTTP context is not cleared.

▼ logo.phtml calls getLogoSrc() / getLogoAlt()

customer_logged_in = false
`parent::getLogoSrc()` → Luma `images/logo.svg`
alt: store's configured alt (empty here)

customer_logged_in = true
`getViewFileUrl('Codilar_PreferenceDemo::images/logo-members.svg')`
alt: “Members”

▼

Page stored in the full-page cache under its own key and returned

Why not `$customerSession->isLoggedIn()` **?** Because of the depersonalisation step above, on every cacheable page the session is already empty when the logo renders, so it would always say “guest”. And even if it worked, the first visitor's logo would be cached and shown to everyone. The HTTP context avoids both problems.

4. Module structure

app/code/Codilar/PreferenceDemo/

```
PreferenceDemo/
├── registration.php                    registers the module with Magento
├── etc/
│   ├── module.xml                      module name + load order
│   └── frontend/
│       └── di.xml                      the preference (storefront only)
├── Block/
│   └── Html/
│       └── Header/
│           └── Logo.php                our replacement logo block
└── view/
    └── frontend/
        └── web/
            └── images/
                └── logo-members.svg    the Members logo image
```

File	Responsibility
`registration.php`	Tells Magento that `Codilar_PreferenceDemo` lives in this folder.
`etc/module.xml`	Declares the module and loads it after Magento_Theme and Magento_Customer.
`etc/frontend/di.xml`	The preference: core Logo block → our Logo block.
`Block/Html/Header/Logo.php`	Extends the core block; returns the Members logo and alt text for logged-in customers.
`view/frontend/web/images/logo-members.svg`	Static image served from `pub/static`.

The folder path of the block mirrors the core class (`Block/Html/Header/Logo`). This is a convention, not a requirement, but it makes the relationship obvious.

5. File-by-file code explanation

5.1 registration.php

app/code/Codilar/PreferenceDemo/registration.php

```
1 <?php
2 use Magento\Framework\Component\ComponentRegistrar;
3
4 ComponentRegistrar::register(ComponentRegistrar::MODULE, 'Codilar_PreferenceDemo', __DIR__);
```

- **Line 4** – `ComponentRegistrar::register()` records that the component of type `MODULE` named `Codilar_PreferenceDemo` is located in `__DIR__` (this folder).
- Composer's autoloader loads every `registration.php` under `app/code` at bootstrap. This is how Magento discovers the module.
- The name must follow `Vendor_Module` and match `etc/module.xml`.

5.2 etc/module.xml

app/code/Codilar/PreferenceDemo/etc/module.xml

```
1  <?xml version="1.0"?>
2  <config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3          xsi:noNamespaceSchemaLocation="urn:magento:framework:Module/etc/module.xsd">
4      <module name="Codilar_PreferenceDemo">
5          <sequence>
6              <module name="Magento_Theme"/>
7              <module name="Magento_Customer"/>
8          </sequence>
9      </module>
10 </config>
```

- **Line 4** – the module name. After `bin/magento module:enable` it appears in `app/etc/config.php`.
- **Lines 5–8** – `<sequence>` sets **load order**:
  - `Magento_Theme` owns the Logo block we replace. Loading after it means our di.xml is merged after Theme's.
  - `Magento_Customer` provides `Customer\Model\Context` and the plugin that writes `customer_logged_in` into the HTTP context.
- If two modules declare a preference for the same class, the one merged **last** wins. `<sequence>` is the tool for controlling that order.

5.3 etc/frontend/di.xml

app/code/Codilar/PreferenceDemo/etc/frontend/di.xml

```
1  <?xml version="1.0"?>
2  <config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3          xsi:noNamespaceSchemaLocation="urn:magento:framework:ObjectManager/etc/config.xsd">
4      <!-- "Whenever Magento asks for the core header Logo block, give it our Logo instead."
5           Declared in etc/frontend so the admin is not affected.
6           Compare the logo at the top of any storefront page as a guest and when logged in. -->
7      <preference for="Magento\Theme\Block\Html\Header\Logo"
8                  type="Codilar\PreferenceDemo\Block\Html\Header\Logo"/>
9  </config>
```

- **Location** – `etc/frontend/` limits the preference to the storefront area. The admin, REST, GraphQL and cron keep the core class. A file in `etc/` would apply everywhere.

- **Lines 4–6** – a comment saying, in plain words, what the preference does and where to see it.
- **Line 7** – `for`: the class (or interface) that other code asks for. It must match exactly; preferences do not apply to subclasses of the `for` class.
- **Line 8** – `type`: the class Magento creates instead. When the `for` is a class, as here, `type` should extend it so every caller still receives a valid `Logo`.
- Changes to di.xml are cached configuration. Run `bin/magento cache:flush`, plus `setup:di:compile` in production mode.

5.4 Block/Html/Header/Logo.php

app/code/Codilar/PreferenceDemo/Block/Html/Header/Logo.php

```php
<?php
declare(strict_types=1);

namespace Codilar\PreferenceDemo\Block\Html\Header;

use Magento\Customer\Model\Context as CustomerContext;
use Magento\Framework\App\Http\Context as HttpContext;
use Magento\Framework\View\Element\Template\Context;
use Magento\MediaStorage\Helper\File\Storage\Database;
use Magento\Theme\Block\Html\Header\Logo as CoreLogo;

/**
 * Replaces the core header logo block (see etc/frontend/di.xml).
 * Guests get the normal logo from the parent class; logged-in customers get the
 * "Members" logo shipped in view/frontend/web/images.
 */
class Logo extends CoreLogo
{
    private const MEMBERS_LOGO = 'Codilar_PreferenceDemo::images/logo-members.svg';

    public function __construct(
        Context $context,
        Database $fileStorageHelper,
        private readonly HttpContext $httpContext,
        array $data = []
    ) {
        parent::__construct($context, $fileStorageHelper, $data);
    }

    public function getLogoSrc()
    {
        if (!$this->isLoggedIn()) {
            return parent::getLogoSrc();
        }

        return $this->getViewFileUrl(self::MEMBERS_LOGO);
    }

    public function getLogoAlt()
    {
        if (!$this->isLoggedIn()) {
            return parent::getLogoAlt();
        }

        return trim(parent::getLogoAlt() . ' ' . __('Members'));
    }

    /**
     * Uses the HTTP context, not the customer session: on full-page-cached pages Magento
     * clears the session before blocks render, but the HTTP context is kept, and it is
     * part of the page cache key, so guests and members get separate cached pages.
     */
```

```
    private function isLoggedIn(): bool
    {
        return (bool)$this->httpContext->getValue(CustomerContext::CONTEXT_AUTH);
    }
}
```

- **Line 2** – `declare(strict_types=1)` enforces scalar types in this file.
- **Lines 6–10** – imports. `Customer\Model\Context` holds the `CONTEXT_AUTH` constant; `Http\Context` is the request-wide context store. `Template\Context` and `Database` are the parent block's dependencies. The core Logo is imported as `CoreLogo` to avoid a name clash with our own class, which is also called `Logo`.
- **Line 17** – `class Logo extends CoreLogo`. Extending is what makes the preference safe. Every method we don't override (`getLogoWidth()`, `getLogoHeight()`, `_getLogoUrl()`…) keeps working, and the template's calls still succeed.
- **Line 19** – the Members logo as a *module asset path*: `Module_Name::path/inside/web`. Magento resolves it to `view/frontend/web/images/logo-members.svg` (or a theme override, see Q&A).
- **Lines 21–28 – the constructor.**
  - It repeats the parent's parameters (`$context`, `$fileStorageHelper`, `$data`) and adds `$httpContext`. `private readonly` uses PHP 8 constructor property promotion.
  - `array $data = []` stays **last**. The layout passes block arguments (`logoPathResolver`, `logo_size_resolver`) through `$data`, so it must still reach the parent.
  - `parent::__construct($context, $fileStorageHelper, $data)` passes the parent's dependencies on. Forgetting this breaks the block.
- **Lines 30–37 – `getLogoSrc()`.** Guests get `parent::getLogoSrc()`, so the admin-configured logo, the theme logo and all core logic still apply. Logged-in customers get `getViewFileUrl(...)`, which builds the versioned static URL, e.g. `/static/version…/frontend/Magento/luma/en_US/Codilar_PreferenceDemo/images/logo-members.svg`.
- **Lines 39–46 – `getLogoAlt()`.** Guests get the configured alt text. Logged-in customers get it with "Members" appended. `__()` makes "Members" translatable, and `trim()` removes the leading space when the configured alt is empty (as in this store), leaving just "Members".
- **Lines 48–56 – `isLoggedIn()`.** Reads `customer_logged_in` from the HTTP context. The docblock records why the session is not used (section 3). It is `private` because it is a detail of this class, not part of the block's API.
- **No return types on the overrides** – the parent's `getLogoSrc()` / `getLogoAlt()` declare none. PHP lets a child add a return type, but if a future Magento version declared a different one the child would fatally error. Matching the parent signature is the safer choice for preferences.

5.5 view/frontend/web/images/logo-members.svg

The file is the Luma theme's `logo.svg` (copied unchanged) with two elements added just before the closing `</svg>`. The logo's paths are long single lines, so only the added part is shown:

app/code/Codilar/PreferenceDemo/view/frontend/web/images/logo-members.svg (added elements)

```
<!-- … Luma logo paths, viewBox="1 2 148 43" … -->
<rect x="61" y="34.6" width="85.6" height="10.4" rx="5.2" fill="#F3793B"/>
<text x="103.8" y="42.9" text-anchor="middle" fill="#fff"
      font-family="Helvetica Neue, Helvetica, Arial, sans-serif"
      font-size="8.4" font-weight="700" letter-spacing="2">MEMBERS</text>
</svg>
```

- **Same** `viewBox` as the original. The header renders the logo at width 170, so the Members logo occupies exactly the same space and the header does not move when a customer logs in.
- `<rect>` – a rounded pill (`rx` = half the height) placed in the empty band under the LUMA wordmark (x 61–146.6, the width of the word). `#F3793B` is the orange already used in the Luma logo.
- `<text>` – white, bold, centred on the pill (`text-anchor="middle"` at the pill's centre x). It uses system fonts, because an SVG loaded via `<img>` cannot load web fonts.
- **Location** – anything in `view/frontend/web/` is a static asset. In developer mode Magento publishes it to `pub/static` on first request. In production it is deployed with `setup:static-content:deploy`.

6. Tested scenarios

Tested over HTTP against the local storefront (`http://localhost:8080`) with the full-page cache enabled. Each page was requested twice to see a cache MISS followed by a HIT, which developer mode reports in the `X-Magento-Cache-Debug` header.

#	Visitor	Page	Cache	Logo in the HTML
1	Guest	Home	MISS	`…/Magento/luma/en_US/images/logo.svg`
2	Guest	Home	HIT	`images/logo.svg`
3	Guest	Phones category	MISS	`images/logo.svg`
4	Logged-in customer	Home	MISS	`…/Codilar_PreferenceDemo/images/logo-members.svg`, alt “Members”
5	Logged-in customer	Home	HIT	`logo-members.svg`
6	Logged-in customer	Phones category	MISS	`logo-members.svg`
7	Guest again	Home	HIT	`images/logo.svg` – the cached member page is never shown to a guest
8	–	Members logo URL	–	HTTP 200, `image/svg+xml`

Scenarios 2, 5 and 7 are the important ones: they prove that guests and members get **separately cached** pages.

7. Install, configure and test

Install

terminal — from the Magento root

```
php bin/magento module:enable Codilar_PreferenceDemo
php bin/magento setup:upgrade
php bin/magento setup:di:compile                # production mode only
php bin/magento setup:static-content:deploy     # production mode only — publishes the SVG
php bin/magento cache:flush
```

Configure

- **Different Members image** – replace `view/frontend/web/images/logo-members.svg`, or override it from a theme at `app/design/frontend/<Vendor>/<theme>/Codilar_PreferenceDemo/web/images/logo-members.svg` without touching the module. In developer mode, also delete the published copy under `pub/static/frontend/*/*/*/Codilar_PreferenceDemo` so it is republished.
- **Guest logo** – unchanged: Content → Design → Configuration → (store view) → Header → Logo Image.
- **Switch off** – `bin/magento module:disable Codilar_PreferenceDemo` and `cache:flush`. A preference cannot be disabled with an attribute the way plugins and observers can.

Manual test script

1. Open the store in a private window (guest) – normal Luma logo.
2. Sign in – the header now shows the MEMBERS badge. Right-click the logo → Open image: the URL ends in `Codilar_PreferenceDemo/images/logo-members.svg`.
3. Browse a few pages and reload – the Members logo stays.
4. Sign out – the normal logo is back.

Useful checks

terminal

```
php bin/magento dev:di:info "Magento\Theme\Block\Html\Header\Logo" frontend   # shows the preference
php bin/magento dev:di:info "Magento\Theme\Block\Html\Header\Logo"            # global area: core class
# any other module with a preference for the same class?
grep -rn 'for="Magento\\Theme\\Block\\Html\\Header\\Logo"' app/code vendor --include=di.xml
curl -sI http://localhost:8080/ | grep -i x-magento-cache-debug               # HIT / MISS
```

8. Design decisions and pitfalls

HTTP context instead of customer session

The full-page cache stores one copy of a page per cache key. Magento builds that key from the URL plus the **HTTP context** (customer group, login state, currency, store…). Two consequences:

- Anything a block shows differently per visitor must be driven by a value that is in the HTTP context, or the first visitor's version is cached for everyone.
- To stop personal data leaking into cached pages, Magento clears the customer session after the layout is generated, so `$customerSession->isLoggedIn()` is `false` while blocks render.

`customer_logged_in` satisfies both: it is set early by core and it is part of the cache key. This is the same approach core uses, e.g. in `Magento\Customer\Block\Account\AuthorizationLink`, to show “Sign In” vs “Sign Out”.

Extend, then override only what changes

The class extends the core Logo and overrides two methods, calling `parent::` for guests. Copying the whole core class instead would freeze today's core code into the module and miss future fixes.

Adding a constructor dependency safely

The parent's parameters are repeated in the same order, the new one is added before `$data`, and everything is passed to `parent::__construct()`. If Magento adds a parameter to the core constructor in a future version, this constructor must be updated too. That is a known maintenance cost of preferences on concrete classes.

Storefront scope

The admin has its own logo block. Scoping the preference to `etc/frontend` keeps the replacement where it is needed and makes `dev:di:info` output easy to reason about.

Same image size

The Members SVG keeps the original `viewBox`, so the layout's `width="170"` gives identical dimensions and the header does not jump on login.

> **Common preference mistakes**
>
> - Not extending the original class – any method you didn't copy is missing and pages fatally error.
> - Changing the constructor and forgetting `parent::__construct()`, or dropping `$data` – layout arguments are lost.
> - Two modules with a preference for the same class – only the last one loaded wins, silently.
> - Using the customer session in blocks on cacheable pages – it is empty during rendering, and the result may be cached for everyone.
> - Forgetting `cache:flush` / `setup:di:compile` after editing di.xml.
> - Expecting a preference to apply to subclasses of the replaced class – it doesn't; it applies only to the exact requested type.

9. Known limitations

- **Custom admin logo** – if a merchant uploads a different header logo, guests see it, but members still see this module's Luma-based Members SVG. A fuller solution would add a second “Members logo” upload field to the design configuration.
- **One preference wins** – a third-party module or theme with its own preference for `Header\Logo` would override this one (or be overridden by it), depending on load order.
- **Constructor coupling** – a future Magento version that changes the core Logo constructor requires updating ours.
- **Luma-specific artwork** – the SVG is built from the Luma logo. A different theme would need its own Members image (it can override the file from the theme, see section 7).
- **Login state only** – every logged-in customer gets the Members logo. Showing it only to a specific customer group (e.g. VIP) would use `Customer\Model\Context::CONTEXT_GROUP` from the same HTTP context.
- **Static asset caching** – after replacing the SVG, browsers and CDNs may keep the old one until static content is redeployed (the URL contains a deployment version).

10. Expected questions and answers

Preferences in general

Q1. What is a preference in Magento 2?

A `di.xml` rule, `<preference for="A" type="B"/>`, that tells the object manager to create class B whenever A is requested. It is used to bind interfaces to implementations, and to substitute a custom subclass for a core class.

Q2. Where does Magento use preferences itself?

Mostly for interfaces, for example `ProductRepositoryInterface` → `ProductRepository`, or `LoggerInterface` → `Magento\Framework\Logger\LoggerProxy`. Service contracts depend on them.

Q3. What happens if two modules declare a preference for the same class?

The di.xml merged last wins, and the other is silently ignored. Module load order (`<sequence>`) decides which. This is the main reason to avoid preferences on concrete classes when a plugin would do.

Q4. Preference vs plugin vs observer: when do you use each?

Observer: react to an existing event. Plugin: change one public method's input, output or execution, and coexist with others. Preference: bind an interface, or replace a class when you must change protected methods or many methods together.

Q5. Can a preference change protected or private methods?

It can override public and protected methods, because the replacement is a subclass. Private methods of the parent cannot be overridden; only the parent's own code can call them.

Q6. Does a preference also replace subclasses of the original class?

No. It applies only when the exact `for` type is requested. A class that extends the core Logo is unaffected.

Q7. Do plugins on the core class still run after a preference?

Yes. Plugins declared for a type also apply to its subclasses, and our class extends the core Logo. `dev:di:info` lists them under “Plugins” and “Plugins for the Preference”; there are none on Logo today.

Q8. Can you set a preference only for the storefront?

Yes. Put it in `etc/frontend/di.xml` (or `etc/adminhtml`, `etc/webapi_rest`, …). This demo does that, and `dev:di:info … frontend` vs the global area shows the difference.

Q9. How do you see which class Magento will create?

`bin/magento dev:di:info "Class\Name" [area]` prints the preference, the constructor parameters and the plugins.

Q10. How do you switch a preference off?

Disable or remove the module, or declare another preference for the same class in a module loaded later. Unlike plugins and observers, there is no `disabled="true"` attribute.

Q11. What changes do you need after editing di.xml?

`bin/magento cache:flush`. In production mode, also `setup:di:compile`, because DI configuration and generated code are compiled.

Q12. What is a virtual type and how does it differ from a preference?

A virtual type is a named copy of an existing class with different constructor arguments, declared in di.xml without a PHP file. A preference swaps which real class is created. A virtual type never replaces anything on its own.

About this demo

Q13. Which class is replaced, and where does it come from?

`Magento\Theme\Block\Html\Header\Logo`, declared as block `logo` in `Magento_Theme/view/frontend/layout/default.xml` and rendered by `html/header/logo.phtml`.

Q14. Why does the new class extend the core Logo?

So every method we don't change keeps working: width, height, the default logo URL and the layout arguments. Callers still receive a genuine `Logo`. We override just `getLogoSrc()` and `getLogoAlt()`.

Q15. How does the block know whether the customer is logged in?

It reads `customer_logged_in` (`Customer\Model\Context::CONTEXT_AUTH`) from `Magento\Framework\App\Http\Context`. Core's `Customer\Model\App\Action\ContextPlugin` sets that value at the start of every request.

Q16. Why not use `$customerSession->isLoggedIn()`?

On cacheable pages, `Customer\Model\Layout\DepersonalizePlugin` clears the customer session after the layout is generated, before blocks render, so it would always return false. The session is also not part of the page cache key.

Q17. With full-page cache on, how can guests and members see different logos?

The cache key includes the HTTP context's vary string, which contains `customer_logged_in`. Guests and members therefore have separate cache entries, and testing showed a cached guest page never shows the Members logo.

Q18. Why does the constructor repeat the parent's parameters?

PHP requires the child to pass the parent's dependencies to `parent::__construct()`. We add `HttpContext` before `$data` and forward the rest unchanged. `$data` must reach the parent because the layout's block arguments travel in it.

Q19. How is the Members image URL built?

`$this->getViewFileUrl('Codilar_PreferenceDemo::images/logo-members.svg')` resolves the module asset through the current theme's fallback and returns the versioned static URL under `/static/…/frontend/Magento/luma/en_US/`.

Q20. Can a theme change the Members logo without editing the module?

Yes. Put a file at `app/design/frontend/<Vendor>/<theme>/Codilar_PreferenceDemo/web/images/logo-members.svg`. Theme fallback picks it before the module's copy.

Q21. Why is the logic in `getLogoSrc()` and not in a template override?

The demo is about preferences. A template override (theme or layout) could also swap the image, but it would need the login check in the template and would copy the core template. Keeping the logic in the block keeps the template untouched.

Q22. Could this be done with a plugin instead?

Yes. `afterGetLogoSrc()` and `afterGetLogoAlt()` plugins would do it, would coexist with other modules, and would be the more conflict-safe choice in production. The preference is used here to demonstrate class substitution.

Q23. Why is “Members” wrapped in `__()` and `trim()`?

`__()` makes it translatable for other store languages. `trim()` removes the leading space when the store's configured alt text is empty, as it is here, so the result is exactly “Members”.

Q24. Why don't the overridden methods declare return types?

The core methods declare none. Keeping the same signature avoids a fatal error if a future Magento version adds a different return type to the parent.

Q25. Why does the header not move when a customer logs in?

The Members SVG keeps the original `viewBox`, and the layout renders both at width 170. The badge was drawn in the empty space under the LUMA wordmark.

Q26. Is the admin affected?

No. The preference is in `etc/frontend/di.xml`. In the global area `dev:di:info` still reports the core class, and no admin code or configuration was changed.

Q27. What happens if the merchant uploads a new logo in the admin?

Guests see the uploaded logo (via `parent::getLogoSrc()`), while members still see this module's Members SVG. A complete solution would add a separate Members logo setting.

Q28. How would you show the Members logo only to one customer group?

Read `Customer\Model\Context::CONTEXT_GROUP` from the HTTP context and compare it with the group ID. It is also part of the page cache key, so caching stays correct.

Q29. How was it tested?

Over HTTP against the storefront with the full-page cache on: guest and logged-in requests, each loaded twice (MISS then HIT, per `X-Magento-Cache-Debug`), plus a guest after a member, and a check that the SVG URL returns 200 (section 6).

Q30. What would break if another module also had a preference for `Header\Logo`?

Only one would apply, decided by load order. To combine both, one module would extend the other's class and declare the other module in its `<sequence>`, or both would switch to plugins.